Optimisation de Réseaux de Neurones pour la Classification Binaire

Juin 2025

Abstract

Ce rapport présente une étude comparative de plusieurs algorithmes d'optimisation appliqués à un réseau de neurones à une seule couche cachée pour la classification binaire de données non linéairement séparables. Le dataset utilisé est `make_moons` de `scikit-learn`. Les algorithmes étudiés sont: Gradient Descent (GD), Stochastic Gradient Descent (SGD), SGD avec Momentum, et SGD avec Nesterov Accelerated Gradient. Leurs performances sont évaluées selon la précision obtenue sur un ensemble de test.

1 Introduction

Les réseaux de neurones artificiels sont largement utilisés pour résoudre des problèmes de classification, notamment quand les données ne sont pas linéairement séparables. Dans ce TP, nous comparons quatre techniques d'optimisation populaires dans l'entraînement de réseaux de neurones. L'objectif est de mesurer leur performance et rapidité de convergence sur un problème synthétique.

2 Méthodes

2.1 Dataset

Le dataset `make_moons` génère 1000 exemples de deux classes entrelacées. Un bruit gaussien de 0.1 est ajouté pour complexifier la séparation des classes. Les données sont divisées en 80% pour l'entraînement et 20% pour le test.

2.2 Architecture du Modèle

- **Couche d'entrée** : 2 neurones correspondant aux caractéristiques d'entrée.
- **Couche cachée** : 10 neurones avec la fonction d'activation ReLU.
- **Couche de sortie** : 1 neurone avec activation sigmoïde pour produire une probabilité binaire.

La fonction de perte utilisée est l'entropie croisée binaire.

2.3 Algorithmes d'Optimisation

Les algorithmes suivants ont été comparés dans ce TP :

- **Gradient Descent (GD)** : mise à jour à partir du gradient global sur l'ensemble des données.

- **Stochastic Gradient Descent (SGD)** : mise à jour du gradient pour chaque échantillon individuellement.
- **SGD avec Momentum** : accélère l'apprentissage en ajoutant un terme d'inertie basé sur l'historique des gradients.
- **SGD avec Nesterov** : améliore le momentum en anticipant la direction du gradient.
- **Adagrad** : adapte dynamiquement le taux d'apprentissage pour chaque paramètre en tenant compte de l'historique des gradients.
- **RMSprop** : introduit une moyenne glissante des gradients au carré pour atténuer les oscillations.
- **Adam** : combine les principes de Momentum et RMSprop avec des moyennes glissantes des gradients et de leurs carrés.

3 Résultats et Discussion

Méthode	**Convergence**	**Précision Test**
Gradient Descent (GD)	Lente	∼95%
SGD	Instable	∼92%
SGD + Momentum	Rapide	∼96%
SGD + Nesterov	Très rapide	∼96%
Adagrad	Moyenne	∼96%
RMSprop	Rapide	∼97%
Adam	Très rapide	∼98%

Table 1: Comparaison des méthodes d'optimisation sur le dataset `make_moons`.

Les algorithmes adaptatifs (Adagrad, RMSprop, Adam) montrent de bonnes performances. Adam se distingue par sa stabilité et sa rapidité de convergence. RMSprop offre également un bon compromis entre vitesse et précision, tandis qu'Adagrad reste efficace malgré une tendance à ralentir après plusieurs itérations.

Les modèles ont été entraînés durant un nombre fixe d'époques avec un taux d'apprentissage $\eta = 0.01$. Le tableau suivant résume les résultats obtenus :

Méthode	**Convergence**	**Précision Test**
GD	Lente	˜95%
SGD	Instable	˜92%
SGD + Momentum	Rapide	˜96%
SGD + Nesterov	Très rapide	˜96%

Le modèle entraîné avec Nesterov converge le plus rapidement tout en maintenant une excellente précision. Le GD, bien que stable, est significativement plus lent.

4 Conclusion

Ce TP met en évidence l'impact du choix de l'algorithme d'optimisation dans l'entraînement des réseaux de neurones. Les méthodes à base de momentum comme Nesterov sont efficaces pour accélérer l'entraînement. Les algorithmes adaptatifs, en particulier Adam, offrent une convergence rapide et une excellente précision. RMSprop constitue aussi un bon compromis entre stabilité et efficacité. Adagrad reste pertinent bien que son taux d'apprentissage diminue rapidement.

5 Disponibilité du Code

Le code complet de ce TP est disponible dans le fichier Jupyter: `tp_sgd_adam.ipynb`.